



Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 2

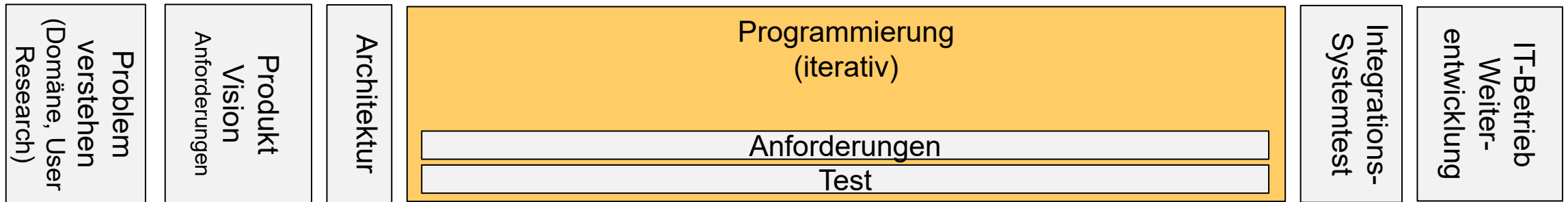
Software-Engineering mit KI

Programmieren ist nicht mehr der Flaschenhals

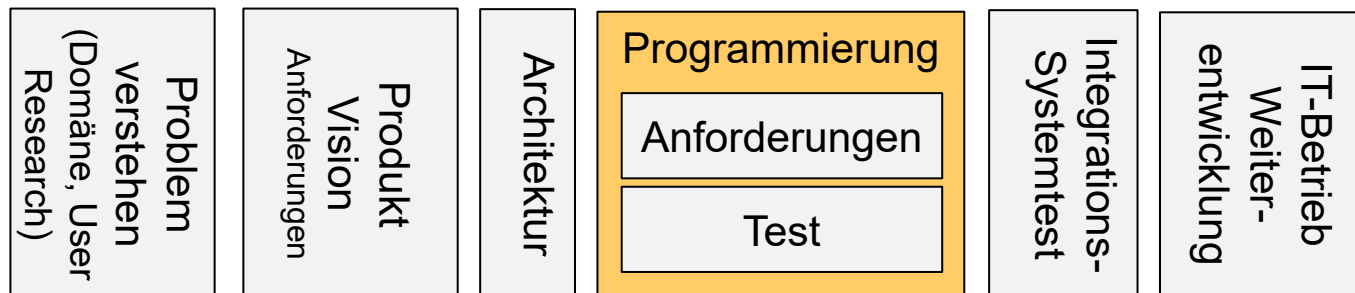
vgl. auch Anthropic: „AI-Native SDLC“

<https://claude.com/blog/the-ai-native-sdlc-playbook>

Agiler Prozess



Prozess mit KI-Agenten (?)





Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 2.1

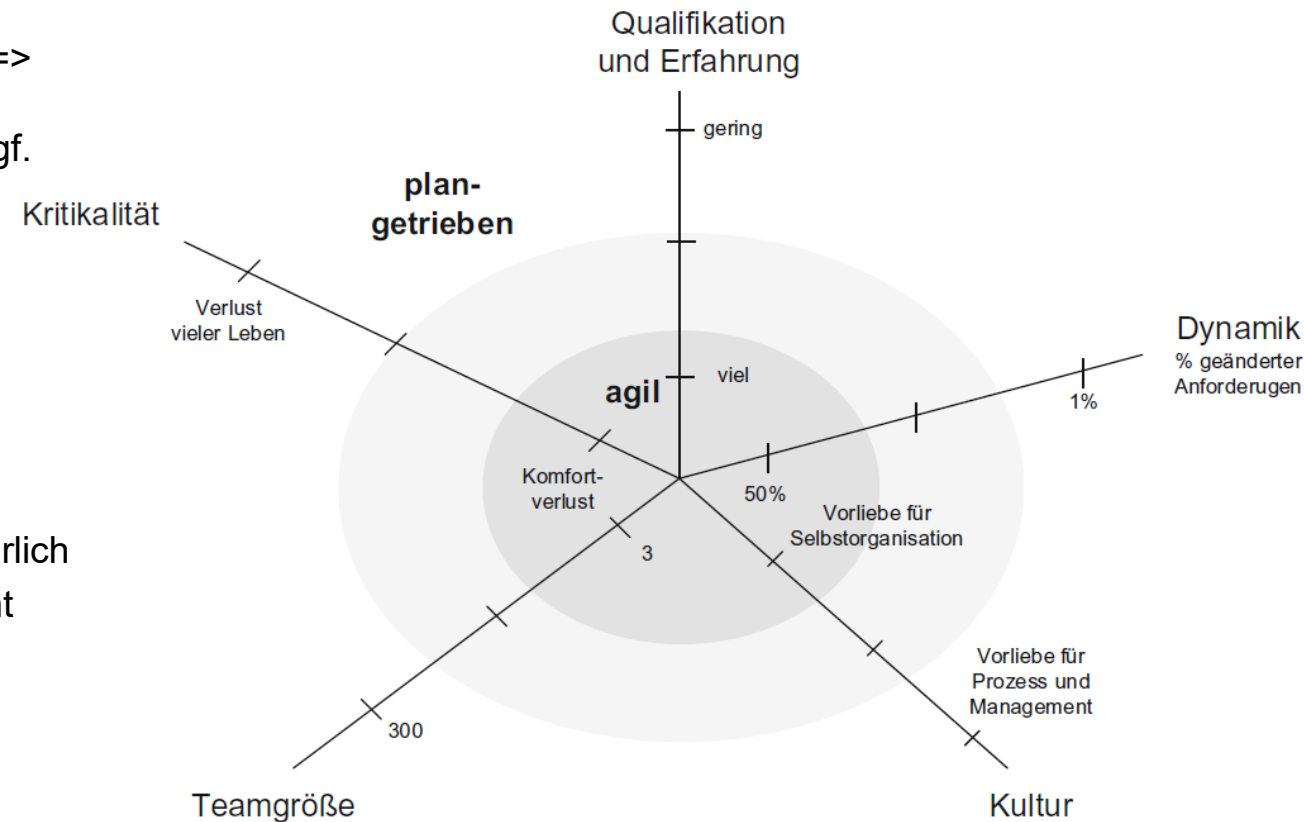
Was genau ist „Software“?

Software ist nicht gleich Software

vgl. Boehm, Turner: Balancing Agility and Discipline

Vorgehen/Prozesse, Werkzeuge hängen von den Eigenschaften der Software und des Teams ab!

- **Kritikalität (Wie teuer ist ein Fehler?)**
 - kritisch => ggf. Verlust von Menschenleben => Standards einhalten (z.B. Funktionale Sicherheit) => Prozess formaler => Fehler ggf. „Fahrlässige Tötung“, Produkthaftung
- **Dynamik (Wie stabil sind Anforderungen?)**
 - Stabil => CR-Verfahren möglich, gut planbar
 - Instabil => agiles Vorgehen, Prototypen, ...
- **Teamgröße**
 - Klein: Kaum Prozesse / Koordination erforderlich
 - Groß: Prozesse und formaleres Management
- **Qualifikation des Teams**
 - Anfänger: Vorgehen nach Kochbuch, viel Feedback
 - Profis: Selbstorganisation
- **Teamkultur**



Software ist nicht gleich Software

Beispiele

Wenn ein Werkzeug oder ein Prozess für ein Dashboard funktioniert, macht das keine Aussage darüber, ob man damit auch Steuergeräte oder ein Transaktionssystem bauen kann!

- **Coder baut Shellskript oder Managerin baut eigenes Dashboard**
 - Eine Entwicklerin, kennt alle Anforderungen (da sie die Software für sich selber baut)
 - Anforderungen überschaubar (Zahl und Komplexität)
 - Trägersystem überschaubar: ein Rechner; Keine oder nur wenige einfache Schnittstellen
- **GUI-Prototyp**
 - Ein / wenige Entwickler, Anforderungen vom Benutzer oder vom Auftraggeber
 - Anforderungen überschaubar, Details noch unwichtig, eher geringe Komplexität, wenige Stakeholder
 - Trägersystem überschaubar, Durchsatz, Verfügbarkeit, Betriebbarkeit, IT-Security etc. eher unwichtig
 - Architektur noch nicht wichtig, Prototyp wird weggeworfen
- **Transaktionssystem einer Bank, Schaden-System einer Versicherung, ...**
 - Mehrere / viele Entwicklerinnen, ggf. längere Historie, Anforderungen vom Fachbereich
 - Komplexe Anforderungen (z.B. „Fahren ab 17-Tarife rechnen“), viele Sonder- und Randfälle, viele Stakeholder, die mitreden
 - Komplexe Architektur, da gewachsene, heterogene Systemlandschaft mit vielen Nachbarn, Durchsatz, Verfügbarkeit wichtig
 - Langlebig, Investitionsgut, Bugfixing leicht
- **Steuergerät in einer Fahrzeugserie**
 - Viele Entwicklerinnen, ggf. längere Historie, Anforderungen aus dem Maschinenbau, Elektrotechnik, Marketing, ...
 - Komplexe Anforderungen, da Sicherheitskritisch, Steuergerät in komplexem Fahrzeugnetz, Feature-Interaction, ...
 - Komplexe Architektur, Steuergerät in komplexem Fahrzeugnetz, Robustheit, Fehlertoleranz wichtig
 - Langlebig, Bugfixing schwer (Rückrufaktion)



Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 2.2

Was genau ist „Engineering“?

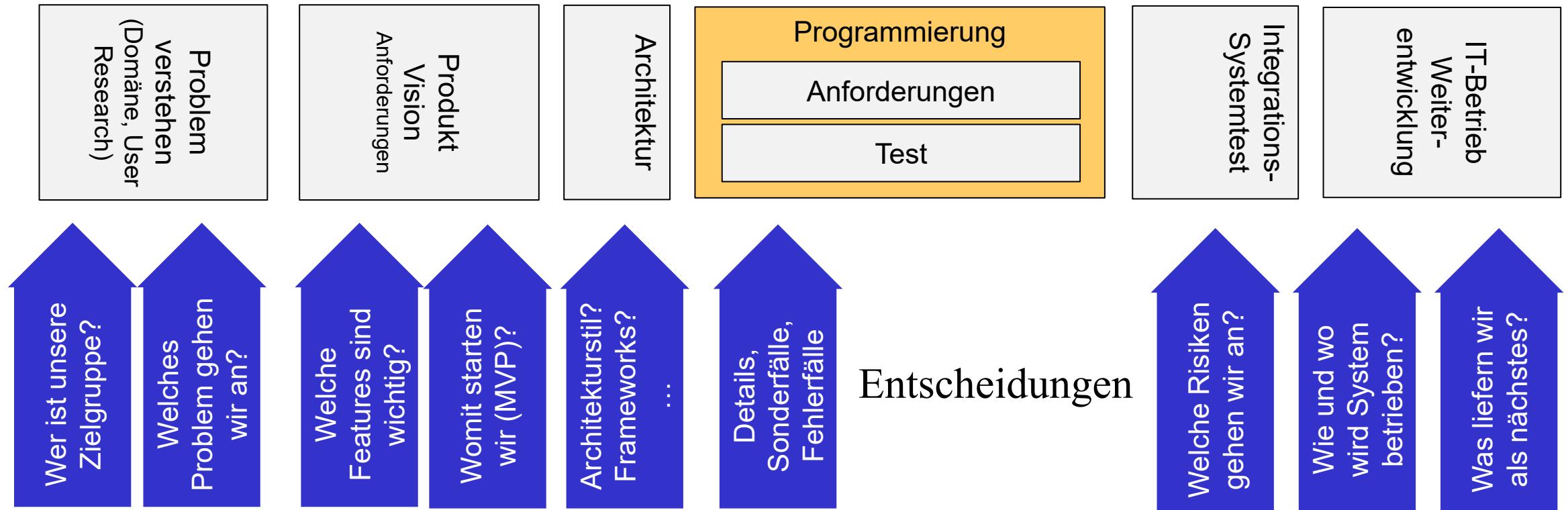
Engineering: Die einsame Ingenieurin auf dem Berg?



Kann man **alleine** Software-Engineering betreiben?

- Ja: Wenn man Software für sich selber baut
(= Anwendungsfall in den meisten Demos im Netz)
- Jein: Wenn bei interner Anwendungsentwicklung, wenn man sich gut auskennt.
- Nein: Wenn man Software für einen Auftraggeber baut
(= Dafür wird man als Ing. bezahlt)

Software-Engineering = Menschen entscheiden



Entscheidungen, die von den Stakeholdern des Produkts getroffen werden!
Stakeholder müssen Inhalt und Konsequenzen ihrer Entscheidungen verstehen

Software-Engineering mit KI-Agenten wie Claude Code

KI entscheidet alles vs. Menschen entscheiden alles

Dashboard
für eine
Managerin
(lokal auf Laptop,
eine Stakeholderin)

Prototyp für
Produkt
(Wegwerf-Prototyp,
anfänglich wenige
Stakeholder)

Transaktions-
system in Cloud
(langlebig, hohe Last,
viele Stakeholder)

Steuergerät
im Auto
(sicherheitskritisch,
komplexes Bordnetz)

Was kostet eine **falsche** Entscheidung?
Entscheidung reversibel?

KI entscheidet

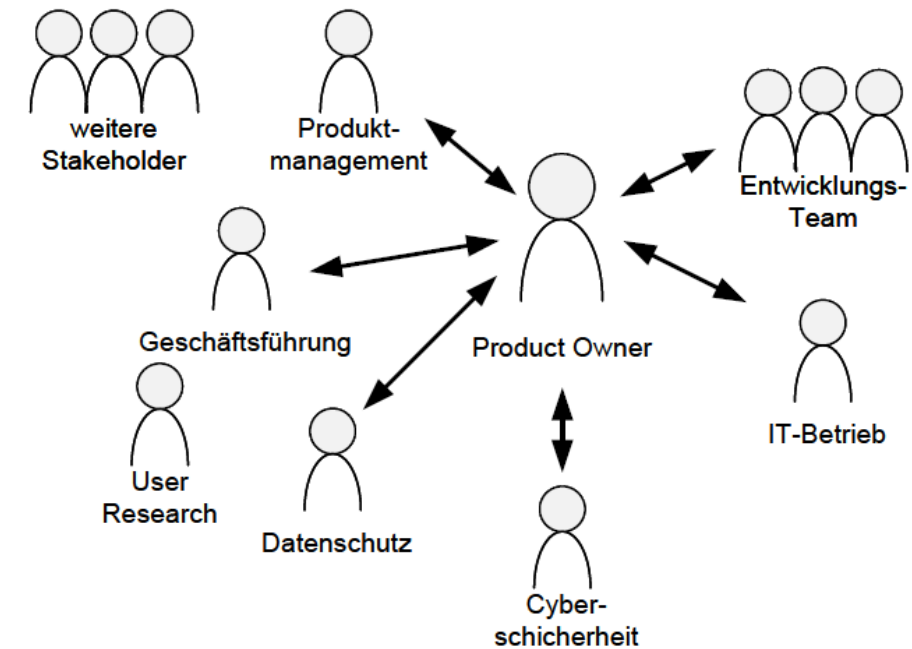
(= KI schlägt vor,
Entscheidung wird
Nicht hinterfragt)

**Menschen
entscheiden**

(= Zeitinvestition
in Entscheidung)

Software-Engineering = Menschen kommunizieren (im Team)

- Stakeholder und Benutzer beteiligen
 - **Anforderungen und Bedürfnisse können nur von ihnen kommen** (=weder die KI noch wir sollten was ausdenken)
 - **Arbeitsteilige Gesellschaft:** Stakeholder wollen idR. nicht selbst mit der KI reden, genauso wenig wie sie ihren Wasserrohrbruch oder ihr Auto selber reparieren wollen
- Informationen beschaffen und konsolidieren
 - z.B. Gemeinsames Verständnis eines Geschäftsprozesses (=> Event Storming **Workshop**)
 - Z.B. MVP und erste Releases entscheiden (=> User Story Mapping **Workshop**)
- **Entscheidungen in der Gruppe** herbeiführen
 - Z.B. Features des MVP? Welche Anforderungen genau?
 - Z.B. Welcher Cloud-Provider, welcher PaaS-Service?
- **Stakeholder müssen Inhalt und Konsequenzen ihrer Entscheidungen beurteilen können**



Software-Engineering = Menschen geben Feedback und Testen

- Agil: Schnelles Feedback => Schnelles Lernen und Anpassen
- **Verantwortung wahrnehmen**
 - Menschen sind für die entstandene Software / das Produkt verantwortlich
[Die KI hat kein Gewissen, sie kann nicht verantwortlich sein]
 - Verantwortung wahrnehmen = Prüfen / Testen / Analysieren / Ausprobieren
[Problem gerade: Mangel an Sorgfalt, Details werden übersehen]
 - QS durch KI => KI generiert Code, Testfall und Testdaten?
[Problem: Sonderfälle werden nicht gesehen, Fehlerfälle rutschen durch, ...]
- **Ist das Problem wirklich gelöst?**
 - Validierung des Produkts mit echten Benutzern, Usability Test
 - Validierung im Pilotbetrieb?
 - Test mit realen Daten (nicht generiert)

Software-Engineering = Informationsverarbeitung

- Informationsgrundlagen für ein Projekt / Produkt
 - Externe Dokumente: Gesetzestexte, Normen, Studien, Statistiken, ...
 - Interne Dokumente: Lastenhefte, IT-Strategie, fachliche Dokumentation, ...
 - Vorhandene Systeme, Nachbarsysteme, inklusive Code und Issues / Wiki
 - Stakeholder: Gelebte Prozesse, Geschäftsregeln, ...
 - Benutzer: Bedürfnisse, Nutzungskontext, ...
- Beispiel: Informationsverarbeitung - Begriffe / Glossar / Domänenmodell
 - Start: Welche Begriffe werden in der Domäne verwendet? Welche Sprache sprechen die Benutzer?
 - Anforderungen: Verwendung der Begriffe zur Formulierung der Anforderungen, Domänenmodell als Ausgangspunkt für Datenmodell, Verwendung der Begriffe in den Dialogen
 - Code: Begriffe finden sich in Klassen, Attributen, Testfällen, Methoden des Codes wieder
 - Test: Begriffe (und Beispiele) als Testdaten
- Informationen müssen über den Produktlebenszyklus / das Projekt hinweg konsistent verwendet werden, z.B. gehört das Glossar in jeden Prompt (Kontext)!



Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 2.3

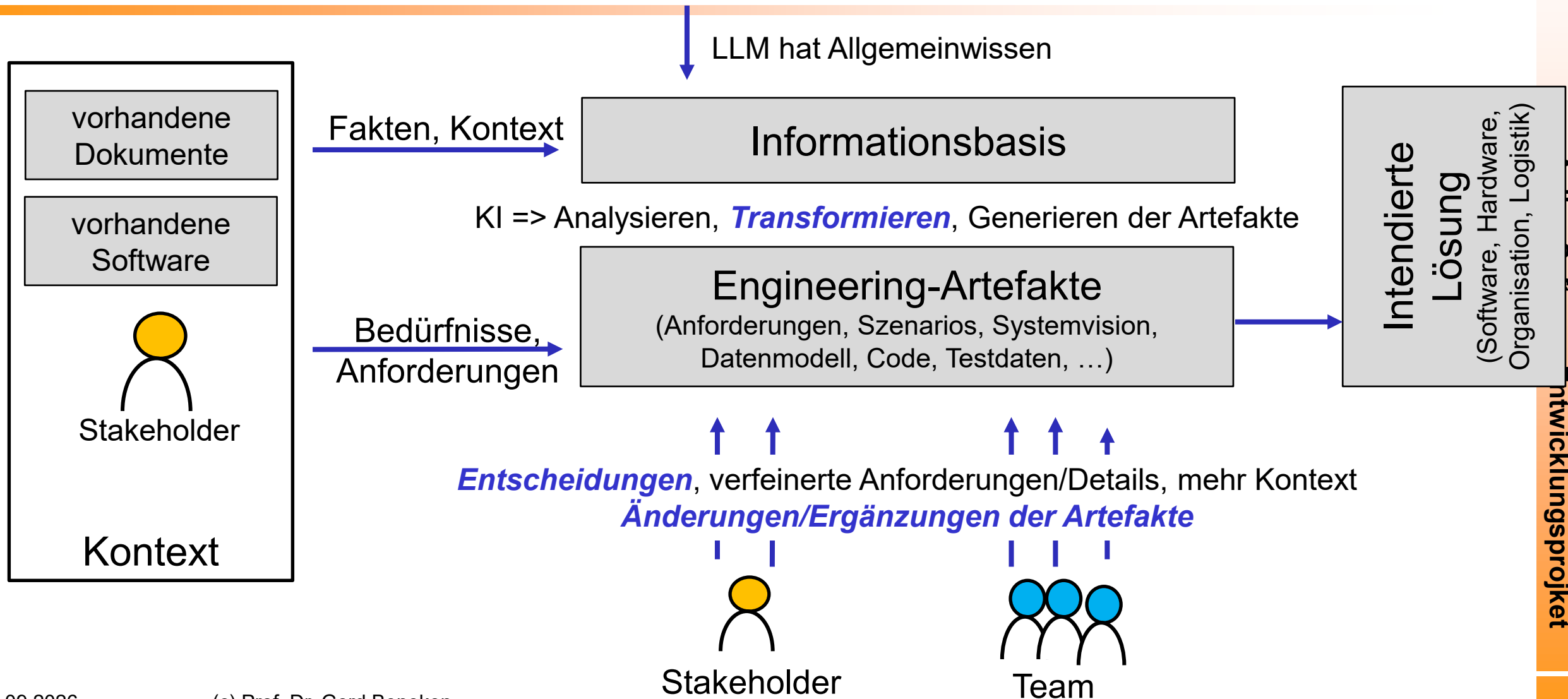
Rollenverteilung Mensch und KI

Kollaboration mit einer KI

1. Intelligente Autovervollständigung
2. KI als Werkzeug: Chat Based Programming, Code Review, ...
3. KI als Sparringspartner: Anforderungen / Architektur diskutieren
4. KI als Kollegin: KI-Agenten wie Claude Code und Codex

KI übernimmt die Informationstransformationen im Projekt

z.B. Interview -> Requirements -> Datenmodell ...



Rollenverteilung

Mensch vs. KI

- Menschen **kommunizieren** mit Menschen
 - Anforderungen / Bedürfnisse erheben, validieren, priorisieren
 - Produkt mit Stakeholdern validieren
- Menschen **entscheiden** gemeinsam
 - Welches Problem lösen? Für wen wird das Produkt gebaut?
 - Welche Features hat das Produkt (zuerst)?
 - Welche Architektur hat das Produkt?
(Cloud-Provider, PaaS, Frameworks, Bibliotheken, ...)
 - Wann wird was an wen geliefert?
- Menschen **planen**
 - Reihenfolge, in der Features gebaut werden
 - Wer macht was (Team <-> KI)
 - Code-Architektur und Implementierungsplan
- Menschen **beurteilen**
 - holen Feedback vom AG / Kunden ein, löst das Ergebnis wirklich das gegebene Problem?
 - sind KI-Ergebnisse gut genug, geeignet, angemessen?
- Menschen tragen **Verantwortung**, sie
 - **Haften für Fehler beim Kunden**,
 - testen, um Fehler beim AG/Kunden zu verhindern
- KI **führt aus**: Pläne / Entscheidungen
 - erstellt Code und Unit-Testfälle
 - erstellt Anforderungs- und Architekturdoku (ADRs)
- KI **hilft** beim Entscheiden
 - beschafft Informationen, arbeitet Informationen passend auf (= Entscheidungsvorlage, PoCs)
 - unterstützt beim Verfeinern, findet Informations- und Entscheidungslücken und Widersprüche
 - macht Vorschläge für Entscheidungen
 - unterstützt beim Planen
- KI **hilft** beim Planen
 - erstellt Detail-Pläne (z.B. Issues)
- KI **hilft** beim Beurteilen
 - Feedback zu Dokumenten und zu Code (Review)
 - Diskussion mit KI über Sachverhalte
- **KI trägt KEINE Verantwortung**



Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 2.4

Ablauf Software-Engineering

Software-Engineering Ablauf (iterativ)

1. Problem finden, analysieren, verstehen, genau beschreiben
 - Domäne: z.B. Welche Geschäftsprozesse werden gelebt? Was läuft nicht optimal?
 - Benutzer: z.B. Welche User sind interessant? Welche Bedürfnisse haben sie?
 - Physik: z.B. Regelungstechnik umsetzen
 - Rahmenbedingungen: z.B. Welche Gesetze müssen eingehalten werden?
2. Anforderungen: (fachlichen) Lösungsvorschlag erarbeiten (iterativ)
 - Produktvision: Wesentliche Features entscheiden, Zielgruppe entscheiden, ...
 - Anforderungen verfeinern (Anwendungsfälle, Akzeptanzkriterien, Beispieldaten)
 - Fachliches Datenmodell erarbeiten
 - Entwurf der Schnittstellen zum Benutzer, z.B. Papierprototyp, Wireframe, ...
3. Architektur: Technischen Lösungsvorschlag erarbeiten
 - Architekturtreiber spezifizieren (Qualitätsanforderungen, Rahmenbed., Risiken)
 - Architekturstil entscheiden, Trägersystem entscheiden
 - Frameworks, Persistenz-Technik entscheiden
 - Prototypen / PoC / Technischer Durchstich
 - Implementierungsplan erarbeiten

Qualitätssicherung, Review, Test

Steuerung: Sind wir auf dem richtigen Weg?

Software-Engineering Ablauf (iterativ)

4. Programmierung

- Offene Fragen bei Anforderungen klären, Testdaten beschaffen
- Testautomatisierung (Modultest)
- Programmieren und testen

5. Integrations- und Systemtest

- Fachlich: Ende-zu-Ende Tests, GUI-Tests, API-Tests
- Technisch: Last- und Stresstest, DAST, ...
- Test in Staging Umgebung (Produktionsnahe Daten / Last)

6. Betrieb und Weiterentwicklung des Systems

- Deployment auf Test, Stage, Prod
- IT-Betrieb technisch: Incident Management, Monitoring, Kostenkontrolle, ...
- IT-Betrieb fachlich: Wie wird das System verwendet? Was machen die Benutzer?
- Weiterentwicklung und Pflege

Qualitätssicherung, Review, Test

Steuerung: Sind wir auf dem richtigen Weg?

Agiles Software-Entwicklungsprojekt